Задание 1.

В целом, набор полей таблицы корректен, однако стоит поменять поле age, обозначающее возраст клиента, на поле `client_birth_date`, отвечающее за дату его рождения. Так как мы заполняем age на стадии добавления пользователя в базу данных, то значение остаётся таким же на протяжении долгого времени, пока мы его каким-либо образом не изменим. Вполне возможно, что нам понадобится наиболее актуальная информация о возрастах клиентов (например, для аналитики), но в случае с полем age в базе будет хранится некорректная информация для пользователей, которых мы добавили ранее, а посчитать возраст клиентов можно будет и через запрос. Поэтому и стоит поменять поле, хранящее возраст клиента на поле с датой его рождения.

Задание 2.

Кажется, стоит завести отдельную от `items` таблицу `item_prices`, где хранить историю изменения цен.

Предлагаемая схема:

Убрать из `items` поле со стоимостью:

```
items (
    item_id number primary key,
    item_name varchar(255)
)
```

Сделать таблицу для учёта цен:

```
item_prices (
    item_id number FOREIGN KEY REFERENCES
items(item_id),
    price number,
    valid_from date,
    valid_to date
```

)

В такой таблице можно будет хранить историю цен для одного и того же товара, информация о которой может понадобиться для составления отчётов.

Задание 3.

В запросе есть следующие ошибки:

- нет группировки по `DEPARTMENT_NAME` (есть группировка `DEPARTMENT_ID` и агрегация `TOTAL_SALARY`);
- некорректная проверка `MANAGER_ID` на равенство `NULL`;
- `WHERE` нельзя использовать для фильтрации групп (`AND SUM(SALARY) >= 100000`);
- я не уверен, но скорее всего лучше связать таблицы `JOIN`-ами, так как в исходном варианте таблицы связаны неявно, вполне возможно берутся не те поля.

Вариант исправленного запроса:

```
SELECT D.DEPARTMENT_ID,
    D.DEPARTMENT_NAME,
    SUM(E.SALARY) AS TOTAL_SALARY
FROM EMPLOYEES E
JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
JOIN LOCATIONS L ON D.LOCATION_ID = L.LOCATION_ID
WHERE E.MANAGER_ID IS NULL
    AND L.CITY = 'SEOUL'
GROUP BY D.DEPARTMENT_ID, D.DEPARTMENT_NAME
HAVING SUM(E.SALARY) > 100000;
```

Задание 4.

Через карандаш не получится перепрыгнуть, если он упал и лежит вплотную

к стене. При попытке прыжка в обычной комнате мы просто ударимся о стену и вряд ли перепрыгнем карандаш.